

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 1
з дисципліни „Професійний Python”
на тему „Організація професійного Python-проєкту та Git.”

Виконала:
студентка 3 курсу
групи Феп-32
Клімашук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

1. Номер і тема лабораторної роботи

Лабораторна робота №1. Організація професійного Python-проєкту та Git.

2. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки (.venv), формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

3. Номер індивідуального варіанта

Варіант №9.

4. Постановка задачі

Відповідно до Варіанта № 9, необхідно розробити консольний Python-застосунок для обліку спортивних результатів.

Сутність (Data Model): Спортсмен / Результат:

- name (ім'я);
- sport_type (вид спорту);
- result (результат);
- category (категорія).

Необхідна функціональність:

- Додавання результату.
- Пошук за видом спорту.
- Визначення найкращого результату.
- Обчислення середнього результату.
- Формування рейтингу.

Вимоги до проєкту:

- Структура src-layout.
- Наявність типів даних (type hints).
- Конструкція if __name__ == "__main__":.
- Файли pyproject.toml, .gitignore, README.md.
- Щонайменше 3 логічні commits у Git.
- Реалізація інтерактивного консольного меню та обробка помилок (завдання підвищеної складності).

5. Структура проєкту

Проєкт побудовано за стандартом src-layout:

- **library_management/** — кореневий каталог проєкту.
 - **.gitignore** — файл із переліком тимчасових файлів та каталогів, що ігноруються системою Git.
 - **pyproject.toml** — конфігураційний файл метаданих та залежностей проєкту.
 - **README.md** — файл із загальним описом проєкту та інструкціями щодо розгортання й запуску.
 - **report/** — каталог для зберігання звіту з лабораторної роботи у форматі PDF чи DOCX.
 - **src/** — каталог джерельного коду (згідно зі стандартом src-layout).
 - **library_management/** — основний пакет програми.
 - **__init__.py** — службовий файл для позначення директорії як Python-пакета.
 - **exceptions.py** — модуль із кастомними класами винятків.
 - **main.py** — точка входу в застосунок та реалізація консольного інтерфейсу.
 - **models.py** — модуль із моделями даних (клас Book).
 - **services.py** — модуль із функціями бізнес-логіки.

6. Опис створених modules

- **src/sports_results/models.py** — містить data-клас AthleteResult із використанням декоратора @dataclass та type hints.
- **src/sports_results/exceptions.py** — містить кастомні винятки (SportsError, AthleteNotFoundError, InvalidResultError) для валідації даних.
- **src/sports_results/services.py** — реалізує бізнес-логіку: додавання, фільтрація, знаходження максимуму, розрахунок середнього значення та сортування для рейтингу.
- **src/sports_results/main.py** — точка входу в застосунок, що містить консольне меню, інтерактивне введення даних та виведення результатів.
- **src/sports_results/__init__.py** — ініціалізаційний файл пакета із зазначенням версії.

7. Текст основних фрагментов коду

models.py

Python

```
from dataclasses import dataclass
```

```
@dataclass
class AthleteResult:
    athlete_id: int
    name: str
    sport_type: str
    result: float
    category: str
```

exceptions.py

Python

```
class SportsError(Exception):
    """Базовий виняток для системи обліку спортивних результатів."""

class AthleteNotFoundError(SportsError):
    """Викликається, коли результат не знайдено."""

class InvalidResultError(SportsError):
    """Викликається при введенні некоректного результату."""
```

services.py

Python

```
from sports_results.exceptions import AthleteNotFoundError, InvalidResultError
from sports_results.models import AthleteResult

def add_result(
    records: list[AthleteResult], name: str, sport_type: str, result: float, category: str
) -> AthleteResult:
    if result <= 0:
        raise InvalidResultError("Спортивний результат повинен бути більше 0.")
    new_id = max([r.athlete_id for r in records], default=0) + 1
    new_record = AthleteResult(new_id, name, sport_type, result, category)
    records.append(new_record)
    return new_record

def find_by_sport(records: list[AthleteResult], sport_type: str) -> list[AthleteResult]:
    s = sport_type.lower().strip()
    return [r for r in records if s in r.sport_type.lower()]
```

```

def get_best_result(records: list[AthleteResult], sport_type: str | None = None) ->
AthleteResult:
    filtered = find_by_sport(records, sport_type) if sport_type else records
    if not filtered:
        raise AthleteNotFoundError("Результатів не знайдено.")
    return max(filtered, key=lambda r: r.result)

def calculate_average_result(records: list[AthleteResult], sport_type: str | None = None) ->
float:
    filtered = find_by_sport(records, sport_type) if sport_type else records
    if not filtered:
        return 0.0
    return sum(r.result for r in filtered) / len(filtered)

def get_rating(records: list[AthleteResult], sport_type: str | None = None) ->
list[AthleteResult]:
    filtered = find_by_sport(records, sport_type) if sport_type else records
    return sorted(filtered, key=lambda r: r.result, reverse=True)

```

8. Пояснення pyproject.toml

Файл описує конфігурацію збірки проєкту за допомогою setuptools та оголошує точкою входу консольну команду sports-results:

Вміст файлу:

```

Ini, TOML
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"
[project]
name = "sports-results"
version = "0.1.0"
description = "Sports Results Management System"
requires-python = ">=3.11"
[project.scripts]
sports-results = "sports_results.main:main"

```

9. Вміст .gitignore

```

Plaintext
.venv/
__pycache__ /
*.pyc
.pytest_cache/
.idea/

```

```
.vscode/  
dist/  
build/  
*.egg-info/  
.env
```

10. Фрагмент README.md

Файл надає стислий опис проєкту та поетапні інструкції для розгортання й запуску:

```
• PS C:\Users\Veronika\Phyton\sports_results> python -m venv .venv  
• PS C:\Users\Veronika\Phyton\sports_results> .venv\Scripts\activate  
• (.venv) PS C:\Users\Veronika\Phyton\sports_results> pip install -e .  
Obtaining file:///C:/Users/Veronika/Phyton/sports_results  
Installing build dependencies ... done  
Checking if build backend supports build_editable ... done  
Getting requirements to build editable ... done  
Preparing editable metadata (pyproject.toml) ... done  
Building wheels for collected packages: sports-results  
Building editable for sports-results (pyproject.toml) ... done  
Created wheel for sports-results: filename=sports_results-0.1.0-0.  
256=0f515e09be84c44bb1dfba7a3c6a0bd0764f22924b21a16843928143c0d370be  
Stored in directory: C:\Users\Veronika\AppData\Local\Temp\pip-ephe  
139819697cebd9cf3c6af44d0551064f45258b5d7adc2fdc0d  
Successfully built sports-results  
Installing collected packages: sports-results  
Successfully installed sports-results-0.1.0  
  
[notice] A new release of pip is available: 24.0 -> 26.2.1  
[notice] To update, run: python.exe -m pip install --upgrade pip  
• (.venv) PS C:\Users\Veronika\Phyton\sports_results> sports-results
```

```
=== УСІ СПОРТИВНІ РЕЗУЛЬТАТИ ===  
ID      Ім'я спортсмена      Вид спорту      Результат      Категорія  
-----  
1      Андрій Шевченко      Біг 100м      10.50      Професіонали  
2      Олена Олексієнко      Біг 100м      11.20      Дорослі  
3      Максим Бондар      Плавання 50м      22.80      Професіонали  
4      Ірина Мельник      Плавання 50м      24.10      Юніори  
5      Дмитро Коваль      Біг 100м      10.80      Дорослі  
=====
```

```
--- СИСТЕМА ОБЛІКУ СПОРТИВНИХ РЕЗУЛЬТАТІВ ---  
1. Показати всі результати  
2. Додати новий результат  
3. Пошук за видом спорту  
4. Визначити найкращий результат  
5. Обчислити середній результат  
6. Сформувати рейтинг спортсменів  
0. Вихід  
Оберіть опцію: █
```

12. Результат git status

```
• (.venv) PS C:\Users\Veronika\Phyton\sports_results> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

13. Результат git log --oneline

```
• (.venv) PS C:\Users\Veronika\Phyton\sports_results> git log --oneline
a7824e0 (HEAD -> main, origin/main) Initial project structure and base configuration
• (.venv) PS C:\Users\Veronika\Phyton\sports_results>
```

14. Відповіді на контрольні запитання

1. Що таке Python-проект? — Це сукупність коду, модулів, залежностей, конфігураційних файлів та документації, організованих за єдиною структурою для вирішення конкретного завдання.
2. Чим проект відрізняється від одного Python script? — Скрипт — це один файл .py. Проект має чітку модульну структуру, ізольоване середовище, конфігурацію пакетування, документацію та контроль версій.
3. Що таке Python module? — Це окремий .py файл, який містить код (функції, класи), призначений для повторного використання.
4. Що таке Python package? — Директорія, яка містить модулі та файл __init__.py.
5. Для чого потрібний __init__.py? — Позначає директорію як Python package і дозволяє здійснювати імпорт з неї.
6. Що таке src-layout? — Структура проекту, де сирцевий код розробки розміщується в окремій директорії src/.
7. Які переваги має src-layout? — Запобігає випадковому імпорту незаінстальованого коду під час тестування та чітко відокремлює код програми від додаткових службових файлів.
8. Для чого використовується virtual environment? — Для ізоляції залежностей та бібліотек одного проекту від інших проектів або системного Python.
9. Як створити .venv? — За допомогою команди `python -m venv .venv`.
10. Як активувати virtual environment? — у Windows: `.venv\Scripts\activate`; у Linux/macOS: `source .venv/bin/activate`.
11. Для чого використовується pip? — Для встановлення та управління сторонніми пакетами й бібліотеками Python.
12. Чому рекомендується `python -m pip`? — Гарантує, що pip запускається саме з того інтерпретатора Python, який активний у даному віртуальному середовищі.
13. Для чого потрібний `pyproject.toml`? — Є єдиним стандартом конфігурації проектів у Python (метадані, збірка, скрипти).
14. Що містить секція `[project]`? — Назву проекту, версію, опис, вимоги до версії Python, залежності та точки запуску скриптів.
15. Що таке `editable installation`? — Встановлення пакета через `pip install -e .`, що дозволяє відразу тестувати зміни у коді без повторної інсталяції.

16. Для чого використовується `if __name__ == "__main__":`? — Для того, щоб блок коду запускався тільки при безпосередньому виконанні файлу як головного скрипта, а не при імпорті.
17. Що таке Git? — Розподілена система контролю версій.
18. Що таке repository? — База даних Git, яка зберігає всі файли проєкту та повну історію їхніх змін.
19. Що таке commit? — Фіксований збережений стан файлів проєкту в історії Git.
20. Для чого використовується `git status`? — Показує поточний стан робочого дерева (змінені, невідстежувані або підготовлені файли).
21. Для чого `git add`? — Додає зміни до області підготовки (staging area) перед створенням комміту.
22. Для чого `git commit`? — Фіксує підготовлені зміни в історії версій із пояснювальним повідомленням.
23. Що показує `git log`? — Історію виконаних коммітів у репозиторії.
24. Для чого потрібен `.gitignore`? — Вказує файли й директорії, які Git не повинен відстежувати (кеш, віртуальні середовища, секрети).
25. Чому `.venv` не потрібно комітити? — Оскільки воно містить системно-залежні бінарні файли великого розміру, а всі необхідні залежності описуються у конфігурації проєкту.
26. Для чого потрібний README? — Надає опис проєкту, інструкцію зі встановлення, запуску та інформацію про авторів.
27. Що таке type hints? — Вказівка типів даних у коді для покращення його читабельності та автоматичної перевірки аналізаторами.
28. Для чого програму розділяють на modules? — Для забезпечення структурованості, легкості читання, спрощення тестування та повторного використання коду.
29. Що таке business logic? — Основна логіка обробки даних, розрахунків та правил роботи програми, відокремлена від інтерфейсу користувача.
30. Чому потрібно відокремлювати application entry point від business logic? — Це дозволяє змінювати або замінювати інтерфейс (наприклад, з консольного на веб або GUI) без зміни основної логіки програми.

15. Висновок

У ході виконання лабораторної роботи №1 було опановано навички створення професійного Python-проєкту з використанням структури `src-layout`. Налаштовано ізольоване середовище `.venv`, створено конфігураційний файл `pyproject.toml`, реалізовано модульну структуру з поділом на моделі даних, бізнес-логіку та точку входу. Проєкт додано до системи контролю версій Git із відповідною серією логічних коммітів. Реалізовано функціонал Варіанта №9 («Облік спортивних результатів»).

